

Título: Lectura de datos JSON desde una URL, filtrado y serialización de usuarios

Descripción:

Este programa accede directamente a una URL que ofrece datos en formato JSON:

<https://jsonplaceholder.typicode.com/todos>.

A partir de estos datos, se cargan en un `ArrayList<Usuario>` únicamente los objetos cuyo campo `userId` es un número par y cuyo campo `completed` es `true`.

Los resultados se muestran por consola y posteriormente se serializan en un fichero binario (`usuarios_filtrados.dat`).

Funcionamiento paso a paso:

1. Conexión a la URL:

- Se utiliza `HttpURLConnection` para establecer una conexión HTTP tipo GET con la dirección indicada.
- Se comprueba que el código de respuesta sea 200 (OK) antes de continuar.

2. Lectura del flujo de datos:

- Se lee el contenido de la respuesta línea a línea mediante un `BufferedReader`.
- El texto completo se almacena en un `StringBuilder`.

3. Parseo del JSON:

- Se utiliza la clase `JsonParser` de la librería Gson para convertir la cadena leída en un `JsonArray`.
- Cada elemento del array representa una tarea con los campos `userId`, `id`, `title` y `completed`.

4. Filtrado de datos:

- Se recorren los objetos JSON y se crea una instancia de `Usuario` por cada uno que cumpla las condiciones:
 - `userId` es par.
 - `completed` tiene valor `true`.

- Los objetos válidos se almacenan en un `ArrayList<Usuario>`.

5. Visualización de resultados:

- Se muestran por consola todos los usuarios filtrados con su identificador y título de tarea.
- Se emplea el método `toString()` de la clase `Usuario` para el formato de salida.

6. Serialización:

- Se guarda el `ArrayList` completo en un fichero binario (`usuarios_filtrados.dat`) utilizando `ObjectOutputStream`.
- La clase `Usuario` implementa `Serializable` para permitir su escritura en formato binario.

Gestión de errores:

- Se controlan errores de conexión (`IOException`) y códigos de respuesta HTTP distintos de 200.
- Se capturan excepciones durante la serialización para garantizar un cierre correcto de los flujos.
- Se informa al usuario en consola si la conexión o la lectura fallan.

Aspectos técnicos:

- Uso de la librería **Gson** para parsear el JSON desde la web.
- Conexión HTTP nativa mediante `HttpURLConnection`.
- Estructura de clase `Usuario` serializable para almacenamiento binario.
- Procesamiento completo en memoria del JSON remoto.
- Filtrado y escritura final de resultados en fichero local.
- La ejecución es autónoma, sin necesidad de ficheros previos.

